第四次实验作业参考答案

只是提供参考，并非标准答案，做法有很多！

目录 | CONTENT

1 数据预处理

2 模型

3 训练与调参

4 结果评估及分析

5 实验报告

基础：数据预处理

3.1 数据预处理

(1) 文本规范化与分词

对每条句子先进行小写化与去首尾空白，再采用基于空格的词级切分。

(2) 词表构建（仅训练集）

分别为源语言（EN）与目标语言（DE）构建独立词表，统计训练集 token 频次并设定最小词频阈值 $f_{min} = 2$，包含固定的特殊符号集合：$S = \{\langle pad\rangle, \langle sos\rangle, \langle eos\rangle, \langle unk\rangle\}$，并保证其索引在词表最前。对低频词与未登录词统一映射到 $\langle unk\rangle$。

(3) 数值化（加入序列边界符）

将句子 token 序列映射为 id 序列，并在首尾分别添加起始与终止标记：

$$x = [\langle sos\rangle, x_1, \dots, x_T, \langle eos\rangle], \qquad y = [\langle sos\rangle, y_1, \dots, y_{T'}, \langle eos\rangle].$$

(4) 动态 padding 与批处理

同一 batch 内对源/目标序列分别进行 padding，使其长度对齐到 batch 内最大长度，padding 值为 $\langle pad\rangle$的索引，训练时损失函数对$\langle pad\rangle$位置进行忽略，避免 padding 影响梯度。

```python
def tokenize(text):
    """
    简单的分词函数（小写 + 按空格分割）

    Args:
        text: str 输入文本

    Returns:
        tokens: List[str] 分词后的 token 列表
    """
    return text.lower().strip().split()
```

```python
def build_vocab(dataset, key, min_freq=2):
    """
    构建词表（仅基于训练集，避免数据泄露）

    Args:
        dataset: 数据集（训练集）
        key: 字段名（"en" 或 "de"）
        min_freq: 最小词频阈值（低于此频率的词会被映射到 <unk>）

    Returns:
        vocab: 词表对象
    """
    counter = Counter()
    for x in dataset:
        counter.update(tokenize(x[key]))
    v = build_vocab_from_counter(
        counter,
        specials=[PAD, SOS, EOS, UNK],
        min_freq=min_freq,
    )
    # 设置默认索引为 UNK（OOV 词会被映射到 <unk>）
    v.set_default_index(v[UNK])
    return v
```

```python
def numericalize(text, vocab):
    """
    将字符串转为模型可处理的索引序列

    格式: [SOS] + tokens + [EOS]

    Args:
        text: str 输入文本
        vocab: 词表对象

    Returns:
        ids: List[int] token 索引列表
    """
    return (
        [vocab[SOS]] +
        [vocab[token] for token in tokenize(text)] +
        [vocab[EOS]]
    )


def collate_fn(batch, src_vocab, tgt_vocab):
    """
    批量数据处理函数（解决变长序列的 padding 问题）

    Args:
        batch: List[Dict] 批次数据，每个元素包含 "en" 和 "de" 字段
        src_vocab: 源语言词表
        tgt_vocab: 目标语言词表

    Returns:
        src: (B, S) 填充后的源序列
        tgt: (B, T) 填充后的目标序列
    """
    src, tgt = [], []
    for x in batch:
        src.append(torch.tensor(numericalize(x["en"], src_vocab)))
        tgt.append(torch.tensor(numericalize(x["de"], tgt_vocab)))

    # 对序列进行 padding，统一批次内长度
    src = torch.nn.utils.rnn.pad_sequence(
        src, batch_first=True, padding_value=src_vocab[PAD]
    )
    tgt = torch.nn.utils.rnn.pad_sequence(
        tgt, batch_first=True, padding_value=tgt_vocab[PAD]
    )
    return src, tgt
```

进阶：针对数据预处理进行的实验

2.2 数据预处理消融实验

为了探究数据表示对模型性能的影响，我采用了控制变量法进行递进式实验。实验分为三个阶段：词频阈值选择、大小写归一化和源序列反转。

2.2.1 词频阈值选择

词频阈值（min_freq）决定了哪些低频词会被替换为<unk>标记。较高的阈值可以减小词表规模、降低稀疏性，但可能丢失语义信息；较低的阈值保留更多词汇，但增加了学习难度。我对比了 min_freq=1、2、5 三种设置，通过验证集 BLEU 分数选择最优配置。选择 BLEU 而非 Token 准确率的原因在于，机器翻译是开放式生成任务，同一源句可能有多种合理译法，BLEU 通过计算 n-gram 重叠度能更好地评估译文质量。

2.2.2 小写归一化

小写归一化将所有文本转换为小写，可以减少词表规模和数据稀疏性，但可能丢失专有名词等大小写敏感的信息。我在最优词频阈值基础上，分别对 RNN 和 Transformer 测试了是否使用小写归一化。

2.2.3 源序列反转

源序列反转是 Sutskever 等人提出的技巧[11]，通过反转源序列使得源句的开头部分更接近目标句的开头，理论上有助于缓解 RNN 在长距离依赖上的困难。我针对 RNN 模型测试了这一策略的有效性，并从模板化程度、输出长度、内容对齐等角度进行了详细的统计分析。

数有所提升。在此基础上，我对比了三种词频阈值的效果：

model	min_freq	BLEU-1	BLEU-2	BLEU-4
RNN	1	25.43	10.54	4.51
	2	26.78	10.66	5.02
	5	**27.43**	**14.43**	**6.91**
Transformer	1	61.95	48.46	29.71
	2	**63.34**	**50.36**	31.75
	5	62.87	50.26	**32.05**

表 1 不同词频阈值下的验证集 BLEU 分数

结果表明两种模型均在 min_freq=5 时取得最优的 BLEU-4 分数。这是因为较高的词频阈值减小了词表规模（**图 9**），降低了模型需要学习的参数量和数据稀疏性问题，使得模型能够更好地学习常用词的翻译模式。

在确定最优词频阈值后，我进一步测试了小写归一化的影响：

model	配置	BLEU-1	BLEU-2	BLEU-4
RNN	**原始**	**27.43**	**16.43**	**6.91**
	小写	24.54	13.91	4.93
Transformer	原始	62.87	50.26	32.05
	小写	**64.01**	**51.39**	**32.96**

表 2 小写归一化实验验证集 BLEU 分数

有趣的是，两种模型对小写归一化的反应不同：RNN 模型在不进行归一化时表现更好，而 Transformer 模型进行归一化后略有提升。这表明 RNN 较为依赖表层特征（如德语名词大写、句首标识）辅助句法分析，归一化造成的信息丢失对其影响较大。相比之下，Transformer 更受益于词表压缩，其强大的自注意力机制能通过全局上下文解决歧义，因此对大小写特征的依赖度较低。鉴于 RNN 在归一化后的显著损失，为保证实验条件的一致性，后续实验统一采取保留大小写的策略。

- 不仅停留在“出结果”，而是主动尝试分析细节设置对实验结果的影响
- 很好的的科研&工程实践习惯。

目录 | CONTENT

1 数据预处理

2 模型

3 训练与调参

4 结果评估及分析

5 实验报告

RNN 整体架构

```python
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device

        assert encoder.hid_dim == decoder.hid_dim, "Hidden dimensions must match!"
        assert encoder.n_layers == decoder.n_layers, "Layer numbers must match!"
        assert encoder.rnn_type == decoder.rnn_type, "RNN type must match!"

    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = src.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim

        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        state = self.encoder(src)

        input = trg[:, 0]

        for t in range(1, trg_len):
            output, state = self.decoder(input, state)
            outputs[:, t, :] = output
            top1 = output.argmax(1)
            teacher_force = random.random() < teacher_forcing_ratio
            input = trg[:, t] if teacher_force else top1

        return outputs
```

- **Teacher Forcing（教师强制）策略**：一个训练技巧。在预测第 t 个词时，我们有一定概率直接使用目标序列中的**真实词**作为下一步的输入，而不是使用模型上一时刻**预测的词**。这能加快训练早期的收敛速度，防止模型由于一步错而步步错。

RNN Encoder

```python
class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, hid_dim, n_layers, dropout, rnn_type='lstm'):
        super().__init__()
        self.hid_dim = hid_dim
        self.n_layers = n_layers
        self.rnn_type = rnn_type.lower()

        self.embedding = nn.Embedding(input_dim, emb_dim)

        if self.rnn_type == 'lstm':
            self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers, dropout=dropout, batch_first=True)
        elif self.rnn_type == 'gru':
            self.rnn = nn.GRU(emb_dim, hid_dim, n_layers, dropout=dropout, batch_first=True)
        else:
            raise ValueError("rnn_type must be 'lstm' or 'gru'")

        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        # src: [batch_size, src_len]
        embedded = self.dropout(self.embedding(src))

        if self.rnn_type == 'lstm':
            outputs, (hidden, cell) = self.rnn(embedded)
            return (hidden, cell)
        else:
            outputs, hidden = self.rnn(embedded)
            return hidden
```

直接调用即可

RNN Decoder

```python
class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hid_dim, n_layers, dropout, rnn_type='lstm'):
        super().__init__()
        self.output_dim = output_dim
        self.hid_dim = hid_dim
        self.n_layers = n_layers
        self.rnn_type = rnn_type.lower()

        self.embedding = nn.Embedding(output_dim, emb_dim)

        if self.rnn_type == 'lstm':
            self.rnn = nn.LSTM(emb_dim, hid_dim, n_layers, dropout=dropout, batch_first=True)
        elif self.rnn_type == 'gru':
            self.rnn = nn.GRU(emb_dim, hid_dim, n_layers, dropout=dropout, batch_first=True)

        self.fc_out = nn.Linear(hid_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, input, state):
        input = input.unsqueeze(1)
        embedded = self.dropout(self.embedding(input))

        output, new_state = self.rnn(embedded, state)
        prediction = self.fc_out(output.squeeze(1))

        return prediction, new_state
```

- 解码器负责将编码器提供的上下文向量“解压”并翻译成目标序列。
- 单步预测：与 Encoder 一次性处理整个序列不同，Decoder 的 forward 每次只处理 一个时间步 的 Token。
- 状态传递：它接收上一个时间步的 state（隐藏状态）和当前步的 input，输出当前步的预测结果和更新后的状态。
- 线性层：self.fc_out 将 RNN 的隐藏层输出映射到目标语言的词表大小，从而决定下一个词是谁。

Transformer总体架构

```python
class Seq2Seq(nn.Module):
    """…
    def __init__(
        self,
        encoder: Encoder,
        decoder: Decoder,
        device: torch.device,
        bos_idx: int = 1,
    ):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
        self.bos_idx = bos_idx

        assert encoder.d_model == decoder.d_model, "
        assert encoder.n_layers == decoder.n_layers,
        assert encoder.n_heads == decoder.n_heads, "
```

模型分为 Encoder（编码器）和 Decoder（解码器） 两大部分。

```python
def forward(self, src: torch.Tensor, trg: torch.Tensor, teacher_forcing_ratio: float = 1.0) ->
torch.Tensor:
    """…
    batch_size = src.size(0)
    trg_len = trg.size(1)
    vocab_size = self.decoder.output_dim

    outputs = torch.zeros(batch_size, trg_len, vocab_size, device=self.device)

    memory, src_key_padding_mask = self.encoder(src)
    # Fast path: full teacher forcing (parallel decoding)
    if teacher_forcing_ratio >= 1.0:
        logits = self.decoder(trg, memory, src_key_padding_mask=src_key_padding_mask)
        outputs[:] = logits
        return outputs

    input_tok = trg[:, 0]  # usually <bos>
    generated = input_tok.unsqueeze(1)  # [batch, 1]

    for t in range(1, trg_len):
        # Decode current prefix
        logits = self.decoder(generated, memory, src_key_padding_mask=src_key_padding_mask)  #
        [b, t, vocab]
        step_logits = logits[:, -1, :]  # [b, vocab]
        outputs[:, t, :] = step_logits
        top1 = step_logits.argmax(1)
        teacher_force = random.random() < teacher_forcing_ratio
        next_tok = trg[:, t] if teacher_force else top1
        generated = torch.cat([generated, next_tok.unsqueeze(1)], dim=1)

    return outputs
```

调用 self.encoder(src) 得到 memory。这个 memory 是源句子的语义表示

情况 A：全并行解码（teacher_forcing_ratio >= 1.0）
逻辑：模型假设下一步总是能拿到正确的答案。
实现：直接将整个 trg（目标序列）丢进解码器。由于解码器内部有“因果遮罩（Causal Mask）”，模型在计算第 t 个位置时看不见t+1及之后的词。

情况 B：自回归循环解码（teacher_forcing_ratio < 1.0）
逻辑：模型开始一个时间步、一个时间步地“吐”词。

Teacher Forcing（教师强制） 是一种训练技巧：在预测第 t个词时，不使用模型在 t-1 时刻生成的可能错误的词，而是直接使用训练集里的“正确答案”作为输入。

Positional Encoding

```python
class PositionalEncoding(nn.Module):
    """
    Standard sinusoidal positional encoding (no trainable parameters).
    Output is added to token embeddings.
    """
    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 5000):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        pe = torch.zeros(max_len, d_model)  # [max_len, d_model]
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)  # [max_len, 1]
        div_term = torch.exp(torch.arange(0, d_model, 2, dtype=torch.float) * (-math.log(10000.0) /
        d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer("pe", pe.unsqueeze(0))  # [1, max_len, d_model]

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        x: [batch, seq_len, d_model]
        """
        x = x + self.pe[:, : x.size(1), :]
        return self.dropout(x)
```

由于 Transformer 是并行处理，不自带顺序感。通过 torch.sin 和 torch.cos 构造固定的位置矩阵。

Transformer Encoder

```python
class EncoderLayer(nn.Module):
    """
    Pre-LN Transformer encoder layer:
    x -> LN -> SelfAttn -> Dropout -> +residual -> LN -> FFN -> +residual
    """
    def __init__(self, d_model: int, n_heads: int, pf_dim: int, dropout: float):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.ffn = FeedForward(d_model, pf_dim, dropout)
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        src: torch.Tensor,
        src_key_padding_mask: Optional[torch.Tensor] = None,
    ) -> torch.Tensor:
        # src: [batch, src_len, d_model]
        x = src

        # Self-attention
        q = self.ln1(x)
        attn_out, _ = self.self_attn(q, q, q, key_padding_mask=src_key_padding_mask,
        need_weights=False)
        x = x + self.dropout(attn_out)

        # Feed-forward
        y = self.ln2(x)
        x = x + self.ffn(y)
        return x
```

Pre-LN 优化：在 Multi-head Attention 前先进行 LayerNorm。这种“先归一化再计算”的结构比原版更稳定，是大模型的主流用法。

注意力计算：self.self_attn(q, q, q, ...)。这里 Q=K=V=q，意味着序列中的每个词都在计算它与同句子中其他词的相关性。（QKV分别是什么？）

Transformer Decoder

```python
class DecoderLayer(nn.Module):
    """…
    def __init__(self, d_model: int, n_heads: int, pf_dim: int, dropout: float):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.cross_attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.ffn = FeedForward(d_model, pf_dim, dropout)

        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)
        self.ln3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        trg: torch.Tensor,
        memory: torch.Tensor,
        trg_mask: Optional[torch.Tensor] = None,
        trg_key_padding_mask: Optional[torch.Tensor] = None,
        src_key_padding_mask: Optional[torch.Tensor] = None,
    ) -> torch.Tensor:
        # trg: [batch, trg_len, d_model], memory: [batch, src_len, d_model]
        x = trg

        # Masked self-attention
        q = self.ln1(x)
        self_out, _ = self.self_attn(
            q, q, q,
            attn_mask=trg_mask,
            key_padding_mask=trg_key_padding_mask,
            need_weights=False,
        )
```

- **第 1 阶段：Masked Self-Attention（掩码自注意力）**
  - **目的**：让模型关注已经生成的译文。
  - **关键点**：传入了 `attn_mask=trg_mask`。这个掩码是一个上三角矩阵，确保在预测第 t 个词时，模型看不见 $t+1$ 之后的词。
- **第 2 阶段：Cross-Attention（交叉注意力/对齐）**
  - **逻辑**：`self.cross_attn(q2, memory, memory, ...)`。
  - Q **(Query)**：来自解码器（当前的翻译进度）。
  - K, V **(Key, Value)**：来自编码器的 `memory`（原文的所有信息）。
  - **作用**：这就是“翻译”发生的时刻。模型在问：“根据我目前翻译出的内容 (Q)，我应该去原文的哪些部分 (K, V) 寻找下一个词的信息？”
- **第 3 阶段：FFN**
  - 与编码器一致，进行最后的特征细化和投影。

```python
        x = x + self.dropout(self_out)

        # Cross-attention
        q2 = self.ln2(x)
        cross_out, _ = self.cross_attn(
            q2, memory, memory,
            key_padding_mask=src_key_padding_mask,
            need_weights=False,
        )
        x = x + self.dropout(cross_out)

        # Feed-forward
        y = self.ln3(x)
        x = x + self.ffn(y)

        return x
```

注意计算注意力时 QKV分别是什么

一些学习资料

- Bahdanau Attention（加性注意力）：首次提出 Attention 机制，打破 Encoder 固定向量瓶颈，解码时动态关注源序列不同位置。
  - https://arxiv.org/abs/1409.0473

- Luong Attention（乘性注意力）：提出乘性（Dot-Product）注意力与 Global / Local Attention，为 Transformer 的 QKV 机制奠定基础。
  - https://aclanthology.org/D15-1166

- Attention is all you need：Transformer
  - https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

目录 | CONTENT

1 数据预处理

2 模型

3 训练与调参

4 结果评估及分析

5 实验报告

train.py

```python
def train_epoch(model, iterator, optimizer, criterion, clip, pad_idx):
    model.train()
    epoch_loss, epoch_acc = 0, 0
    for src, trg in tqdm(iterator, desc="  Training", leave=False):
        src, trg = src.to(DEVICE), trg.to(DEVICE)
        optimizer.zero_grad()
        output = model(src, trg)
        output_dim = output.shape[-1]
        loss = criterion(output[:, 1:].contiguous().view(-1, output_dim),
                         trg[:, 1:].contiguous().view(-1))
        acc = calculate_accuracy(output[:, 1:], trg[:, 1:], pad_idx)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
        optimizer.step()
        epoch_loss += loss.item()
        epoch_acc += acc
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

训练代码大同小异
相信在AI的帮助下不是什么大问题

调参数的策略上次作业讲过，不再重复

目录 | CONTENT

1 数据预处理

2 模型

3 训练与调参

4 结果评估及分析

5 实验报告

评估指标介绍及实现

3.1.1. BLEU-1 / BLEU-2 / BLEU-4

我们报告 n-gram 精确匹配的 BLEU-1/2/4 分数。BLEU 采用对整个验证集/测试集汇总后计算平均值。

3.1.2. ROUGE-L

ROUGE-L 基于最长公共子序列（LCS）衡量生成译文与参考译文的序列重叠程度，上反映顺序一致性与覆盖情况。

3.1.3. BERTScore

为衡量语义层面的相似度，我们采用 BERTScore，使用的 backbone 为 `microsoft/deberta-large-mnli`，使用默认参数。一共包含 BERTScore 的 Precision / Recall / F1 三项。即先通过 BERT 编码再做指标计算。

```python
"""
ref_token_lists: List[List[str]]
hyp_token_lists: List[List[str]]
"""
refs = [[r] for r in ref_token_lists]  # corpus_bleu expects list of list of
hyps = hyp_token_lists
# from nltk.translate.bleu_score import corpus_bleu
bleu1 = corpus_bleu(refs, hyps, weights=(1, 0, 0, 0))
bleu2 = corpus_bleu(refs, hyps, weights=(0.5, 0.5, 0, 0))
bleu4 = corpus_bleu(refs, hyps, weights=(0.25, 0.25, 0.25, 0.25))

rouge = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)
ref_strs = [" ".join(r) for r in ref_token_lists]
hyp_strs = [" ".join(h) for h in hyp_token_lists]
rouge_l = sum(
    rouge.score(r, h)["rougeL"].fmeasure
    for r, h in zip(ref_strs, hyp_strs)
) / max(1, len(hyp_strs))

metrics = {
    "BLEU-1": float(bleu1),
    "BLEU-2": float(bleu2),
    "BLEU-4": float(bleu4),
    "ROUGE-L": float(rouge_l),
}
```

```python
from bert_score import score

cands = [
    "The cat sits on the mat.",
    "A man is playing guitar."
]

refs = [
    "A cat is sitting on the mat.",
    "Someone plays a guitar."
]

P, R, F1 = score(
    cands,
    refs,
    model_type="microsoft/deberta-large-mnli",  # 使用指定模型
    lang="en",
    verbose=True
)

print("Precision:", P.tolist())
print("Recall:", R.tolist())
print("F1:", F1.tolist())
print("Average F1:", F1.mean().item())
```

如何计算模型参数量

```python
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

实验的结果如何展示？---》美丽表格而不是终端截图

优秀案例展示：

配置	BLEU-1	BLEU-2	BLEU-4	F1	AverageF1	time(s)
LSTM	27.43	14.43	6.91	0.511	0.734	326.4
GRU	30.21	17.57	6.28	0.578	0.763	294.1
BiLSTM	56.71	41.89	23.29	0.560	0.749	307.2
BiGRU	59.30	44.86	26.06	0.584	0.767	**292.9**
BiGRU+Attention	**65.06**	**52.64**	**33.92**	**0.665**	**0.798**	355.0

表 3 RNN 实验结果

配置	BLEU-1	BLEU-2	BLEU-4	F1	AverageF1	time(s)
ReLU+Sinusoidal	66.598	54.426	36.304	0.705	0.832	494.285
GELU+Sinusoidal	67.199	55.161	36.773	**0.707**	0.832	494.009
GELU+Learned_abs	66.252	53.413	34.845	0.683	0.824	497.860
GELU+T5-RBP	64.049	52.171	34.758	0.685	0.826	528.215
GELU+RoPE	**69.580**	**57.857**	**40.020**	**0.707**	**0.834**	612.745
MHA	69.014	56.994	38.804	0.708	0.834	401.371
MQA	67.508	55.328	37.135	0.694	0.825	**353.818**
GQA	66.926	54.852	36.758	0.701	0.830	363.436

表 4 Transformer 实验结果

这里的指标名字最好写的更明确一些
实验结果的分析，没有特殊要求，体现思考即可

5. 实验结果与分析

考虑到 RNN 解码过程串行性强、Beam Search 推理成本显著高于 Transformer，实验仅在 Transformer Pre-LN 上验证 Beam Search 的增益，用于展示生成策略对高性能模型的进一步提升空间，但出于公平性考虑，表 4 中测试集指标均基于贪心解码策略，对完整测试集进行测试。

表 4：实验结果对比

模型	参数量(M)	BLEU-1	BLEU-2	BLEU-4	ROUGE-L	BERTScore	训练时长(min)	训练轮数
BasicRNN	16.9	0.5648	0.4300	0.2620	0.5686	0.7936	2.99	34
RNNWithAttention	17.5	0.6216	0.4985	0.3311	0.6393	0.8313	10.05	32
BasicTransformer	15.6	0.6335	0.5196	0.3596	0.6405	0.8379	5.27	30
Transformer Pre-LN	15.6	0.6439	0.5326	0.3717	0.6630	0.8443	8.10	47

注：所有指标均针对 Multi30K 英→德翻译任务；BLEU 使用 nltk.translate.bleu_score.corpus_bleu 函数；ROUGE-L 使用 rouge_score 库的 RougeScorer；BERTScore 使用 deberta-large-mnli 模型作为 backbone；训练基于 NVIDIA GeForce RTX3090.

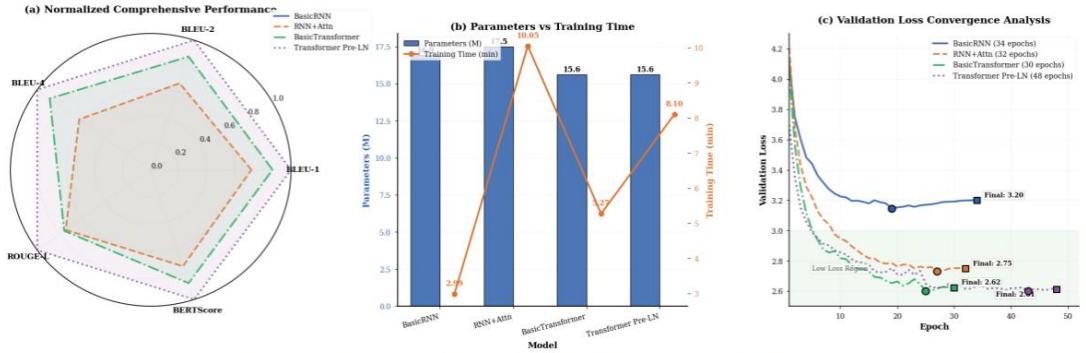

图 14：四个模型综合性能、参数量与训练时间、验证集 loss 可视化

目录 | CONTENT

1 数据预处理

2 模型

3 训练与调参

4 结果评估与分析

5 实验报告

实验报告的结构（进阶要求非硬性要求）

1. 摘要（Abstract）
- 使用了什么模型与方法
- 最佳实验结果
- 模型参数量 / 规模

2. 模型介绍（Model）
- 使用的模型结构 / 预训练模型
- 是否有结构或设置改动

3. 方法与实验（Method & Experiments）
- 使用了哪些方法（loss / 策略 / trick）
- 做了哪些实验（对比、消融、参数、经验性实验）
- 实验如何进行 & 遇到的问题

4. 结果与分析（Results & Analysis）
- 实验结果（表格 / 曲线）
- 对结果的理解与解释

5. 思考与总结（Discussion）
- 主要收获与局限性
- 可改进方向 / 后续工作

- 本案例仅作为参考示例（来自优秀作业）
- 并不是说「只有这样写才是对的」
- 也不是说「按这个结构一定得高分」

- 得分高的原因不是结构本身，而是助教能更清楚地看到你的思考、亮点和实验结果

作业总体情况

分数是怎么算的？完成基础要求给基础分，在此基础上做得好的给加分

代码实现		
model.py	train.py	README
10（模型）	10（实验）	10（送分）

模型性能				
参数	BLEU4	Rouge L F1	Bert F1	SCORE
1(写了就给分)	3（线性插值分）	3（线性差值分）	3（线性差值分）	10

实验报告				创新
结构（基本）	说明（基本）	可视化 (包括性能表格)	思考	
12.5	12.5	12.5	12.5	10

分数分布，可自己对照排名

均分	79.68096516
小于60	4
60-70	18
70-80	10
80-90	21
90-95	17
95-100	10

谢谢聆听

Thank You